

درس سیستم عامل

تمرین عملی چهارم سوال اول

402105727

متین باقری

402105802

فاطمینا تیمارچی

خروجی کامل کد در فایل Q1_output.txt موجود است (چون خروجی خیلی زیاد بود تو عکس جا نشد برای همین خروجی رو در یک فایل متنی قرار دادم) و همچنین کد unthread.c ضمیمه شده است.

برای این تمرین استراکت در فایل pthread.h موجود بود برای همین فقط کافی بود فایل pthread.c رو تغییر بدیم.

توضیح همه توابع در pthread به صورت زیر هست:

1- pthread_init: در این تابع pthread با index صفر به عنوان pthread فعلی انتخاب میشه و وضعیت آن را running قرار میده (علت اینکار هم اینه که قبل از اینکه بیایم pthread های جدیدمون رو بسازیم باید یک pthread فعال داشته باشیم تا scheduler اجرا بشه)

```
void
pthread_init(void)
{
    current_thread = &all_thread[0];
    current_thread->state = RUNNING;
}
```

2- pthread_schedule: این تابع سیستم ما رو زمانبندی میکنه. به این صورت که میاد شماره pthread فعلی رو بر اساس آدرسش پیدا میکنه و سپس با روش round-robin (الگوریتم ساده و منصفانه) میاد pthread بعدی رو در آرایه جست و جو میکنه اگر pthread, runnable بود pthread جدید انتخاب میشه. به عبارت دیگه اولین pthread ای که runnable هست انتخاب میشه و pthread قبلی در prev نخیره میشه و pthread جدید وضعیتش به running تغییر پیدا میکنه و به pthread جدید و context جدید switch میکنه. اگه هم pthread, runnable نبود هم از برنامه خارج میشه. (شرط $(start+i) \% MAX_THREAD$)

```

void thread_schedule(void)
{
    int start = current_thread - all_thread;
    for(int i=1; i<=MAX_THREAD; i++){
        int idx = (start + i) % MAX_THREAD;
        if(all_thread[idx].state == RUNNABLE){
            struct thread *prev = current_thread;
            current_thread = &all_thread[idx];
            current_thread->state = RUNNING;
            thread_switch(&prev->context, &current_thread->context);
            return;
        }
    }

    printf("thread_schedule: no runnable threads\n");
    exit(0);
}

```

3- thread_create : میاد دنبال اولین entry در جدول thread ها میگرده (اولین thread آزاد رو پیدا میکنه) و این thread آماده تخصیص منابع هست و چون نمیخوایم thread های runnable رو overwrite کنیم میایم این thread رو انتخاب میکنیم و استیت این thread جدید رو برابر runnable میزاریم بنابراین این thread آماده اجرا هست و تنها نیاز داره که scheduler این رو انتخاب کنه. (علت اینکه runnable میشه و running نمیشه این هست که اگه Running باشه اون وقت scheduler نمیتونه بفهمه که thread فعلی کدومه و در نتیجه ممنکه در این حالت در اجرای برنامه اختلال ایجاد بشه). حالا که thread مورد نظر رو پیدا کردیم به این صورت عمل میکنیم که میایم یک استک برای thread ها تعیین میکنیم و یک stack pointer به انتهای استک اختصاص میدیم و ra رو برابر تابع thread قرار میدهیم (همون تابع func) و بنابراین thread با اجرای تابع func شروع میشه و sp (stack pointer) رو مقدار دهی میشه. حالا scheduler میخواد thread جدید رو اجرا بکنه پس باید switch کنه پس میاد از CPU از آدرس ra شروع میکنه اجرا میکنه و اگه ra یک thread نباشه در نتیجه نمیفهمه که باید از کجا شروع کنه بنابراین thread مستقیماً وارد func میشه و دیگه نیازی به wrapper نداریم.

```
void
```

```

thread_create(void (*func)())
{
    for(int i = 0; i < MAX_THREAD; i++) {
        if(all_thread[i].state == FREE) {
            struct thread *t = &all_thread[i];
            t->state = RUNNABLE;
            uint64 sp = (uint64)t->stack + STACK_SIZE;
            t->context.ra = (uint64)func;
            t->context.sp = sp;

            return;
        }
    }
    printf("No free thread!\n");
}

```

4- تابع thread_yield: وضعیت thread فعلی از running تبدیل میشه به runnable چون اجرای thread هنوز کامل نشده و فعلا چون دیگه با CPU کاری نداره باید CPU رو ازش بگیرم . در آخر هم تابع thread_schedule صدا زده میشه تا یک thread جدید انتخاب بشه.

```

void thread_yield(void)
{
    if(current_thread->state == RUNNING)
        current_thread->state = RUNNABLE;
    thread_schedule();
}

```

5- تابع thread_a و thread_b و thread_c: برای هر کدام از thread ها یک تابع نوشتم در این تابع علامت شروع thread را فعال میکنه و سپس از 0 تا 99 چاپ میکنه و بعد از هر چاپ تابع thread_yield رو صدا میزنه تا thread های دیگه هم اجرا بشن و در انتها هم thread خودش رو FREE میکنه و به scheduler اعلام میکنه که بره thread بعدی رو اجرا کنه .

```

volatile int a_started, b_started, c_started;
volatile int a_n, b_n, c_n;

void thread_a(void) {
    a_started = 1;
    for(a_n = 0; a_n < 100; a_n++) {
        printf("thread_a %d\n", a_n);
        thread_yield();
    }
    printf("thread_a : exit after 100\n");
    current_thread->state = FREE;
    thread_schedule();
}

void thread_b(void) {
    b_started = 1;
    for(b_n = 0; b_n < 100; b_n++) {
        printf("thread_b %d\n", b_n);
        thread_yield();
    }
    printf("thread_b : exit after 100\n");
    current_thread->state = FREE;
    thread_schedule();
}

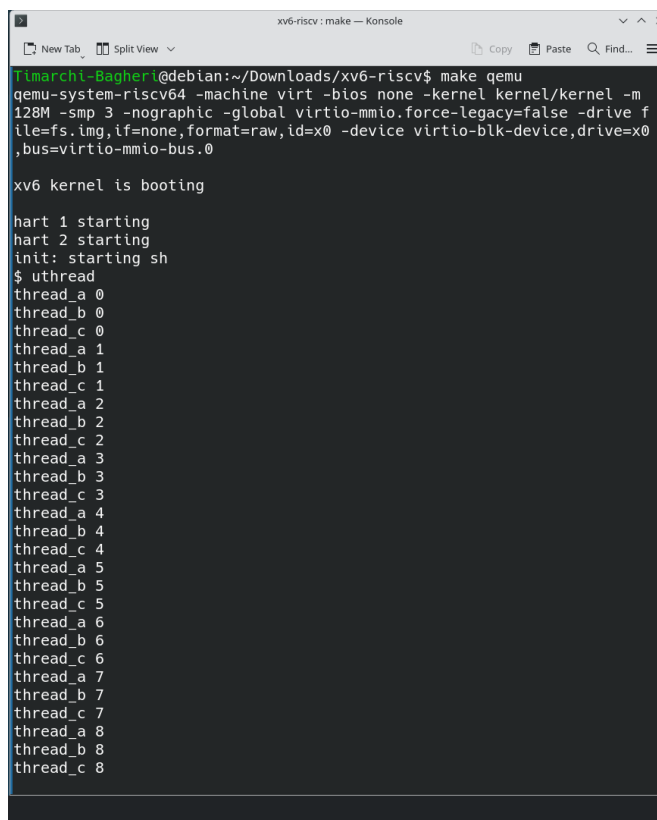
void thread_c(void) {
    c_started = 1;
    for(c_n = 0; c_n < 100; c_n++) {
        printf("thread_c %d\n", c_n);
        thread_yield();
    }
    printf("thread_c : exit after 100\n");
    current_thread->state = FREE;
    thread_schedule();
}

```

6- تابع main : در این تابع هم مقادیر اولیه تنظیم میشه و وضعیت thread صفر را running میکنیم .

```
int
main(int argc, char **argv[])
{
    a_started = b_started = c_started = 0;
    a_n = b_n = c_n = 0;
    thread_init();
    thread_create(thread_a);
    thread_create(thread_b);
    thread_create(thread_c);
    thread_schedule();
    exit(0);
}
```

چون خروجی خیلی طولانی بود در عکس جا نمیشد و برای همین تنها از اول و آخر ترمینال عکس گرفتیم . عکس خروجی ها رو به صورت کامل در فایل Q1_output.txt گذاشتم .



```
xv6-riscv: make - Konsole
Timarchi-Bagheri@debian:~/Downloads/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive f
ile=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0
,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ uthread
thread_a 0
thread_b 0
thread_c 0
thread_a 1
thread_b 1
thread_c 1
thread_a 2
thread_b 2
thread_c 2
thread_a 3
thread_b 3
thread_c 3
thread_a 4
thread_b 4
thread_c 4
thread_a 5
thread_b 5
thread_c 5
thread_a 6
thread_b 6
thread_c 6
thread_a 7
thread_b 7
thread_c 7
thread_a 8
thread_b 8
thread_c 8
```

```
xv6-riscv : make -- Konsole
New Tab Split View Copy Paste Find...
thread_c 88
thread_a 89
thread_b 89
thread_c 89
thread_a 90
thread_b 90
thread_c 90
thread_a 91
thread_b 91
thread_c 91
thread_a 92
thread_b 92
thread_c 92
thread_a 93
thread_b 93
thread_c 93
thread_a 94
thread_b 94
thread_c 94
thread_a 95
thread_b 95
thread_c 95
thread_a 96
thread_b 96
thread_c 96
thread_a 97
thread_b 97
thread_c 97
thread_a 98
thread_b 98
thread_c 98
thread_a 99
thread_b 99
thread_c 99
thread_a : exit after 100
thread_b : exit after 100
thread_c : exit after 100
thread_schedule: no runnable threads
$
```